

Thực hành KTMT Lab 12

Lê Quốc Đăng

20225801

Assignment 1

- Code

```
.eqv IN_ADDRESS_HEXKEYBOARD 0xFFFF0012
.eqv OUT_ADDRESS_HEXKEYBOARD 0xFFFF0014
.data
mess: .asciiz "\n"
.text
main:
li $t1, IN_ADDRESS_HEXKEYBOARD
li $t2, OUT_ADDRESS_HEXKEYBOARD
li $t3, 0x01 # check row 4 with key 0, 1, 2, 3
li $t4, 0x02 # check row 4 with key 4, 5, 6, 7
li $t5, 0x04 # check row 4 with key 8, 9, A, B
li $t6, 0x08 # check row 4 with key C, D, E, F

polling:
sb $t3, 0($t1) # must reassign expected row
lb $a0, 0($t2) # read scan code of key button
bne $a0, $zero, print
sb $t4, 0($t1) # must reassign expected row
lb $a0, 0($t2) # read scan code of key button
bne $a0, $zero, print
sb $t5, 0($t1) # must reassign expected row
lb $a0, 0($t2) # read scan code of key button
```

```

bne $a0, $zero, print
sb $t6, 0($t1) # must reassign expected row
lb $a0, 0($t2) # read scan code of key button
bne $a0, $zero, print
j sleep
print:
li $v0, 34 # print integer (hexa)
syscall
la $a0, mess #print "\n"
li $v0, 4
syscall
sleep:
li $a0, 3000 # sleep 3000ms
li $v0, 32
syscall
back_to_polling:
j polling # continue polling
exit:

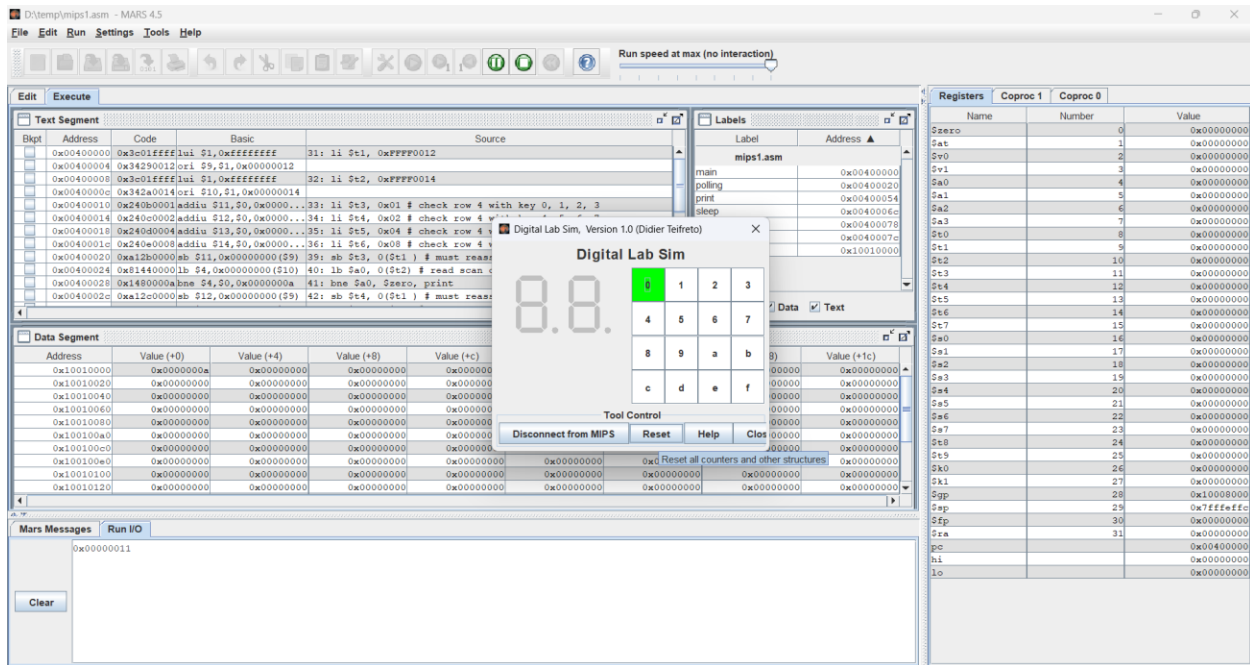
```

- Thực thi

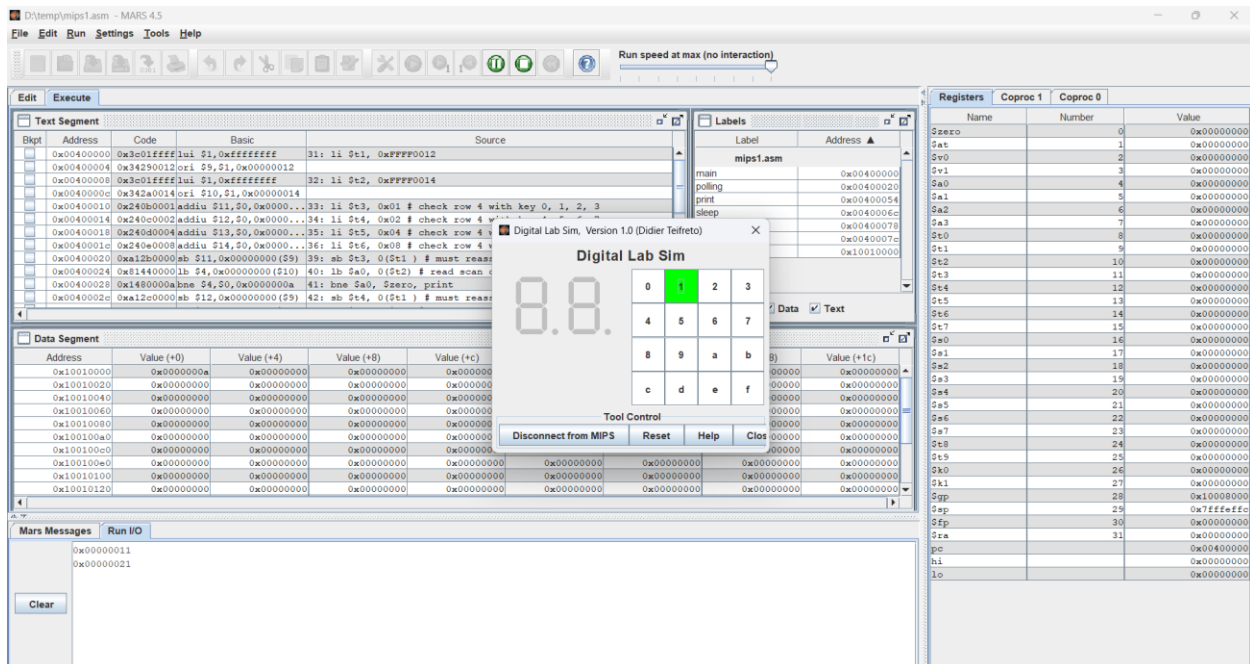
-Chương trình thực hiện lần lượt gán giá trị \$t1 lần lượt bằng 0x01, 0x02, 0x04, 0x08 để kiểm tra xem hàng nào được bấm. Load giá trị \$t2, nếu != 0 thì in ra.

-Kết quả

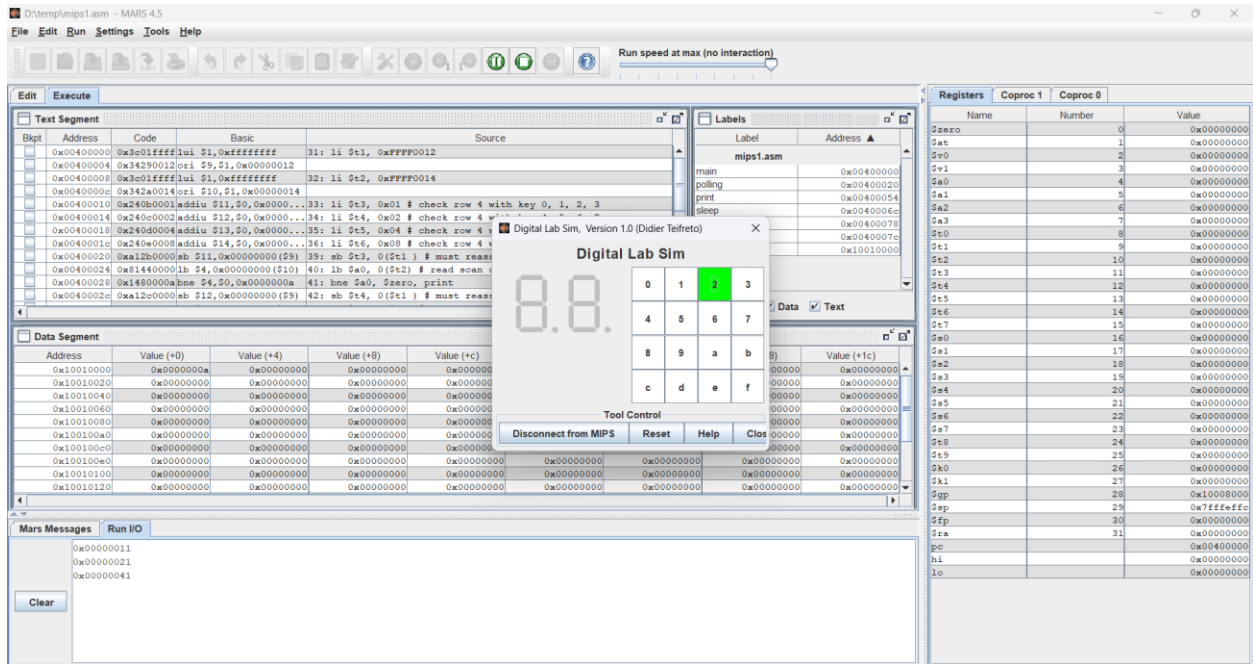
- Bấm phím 0



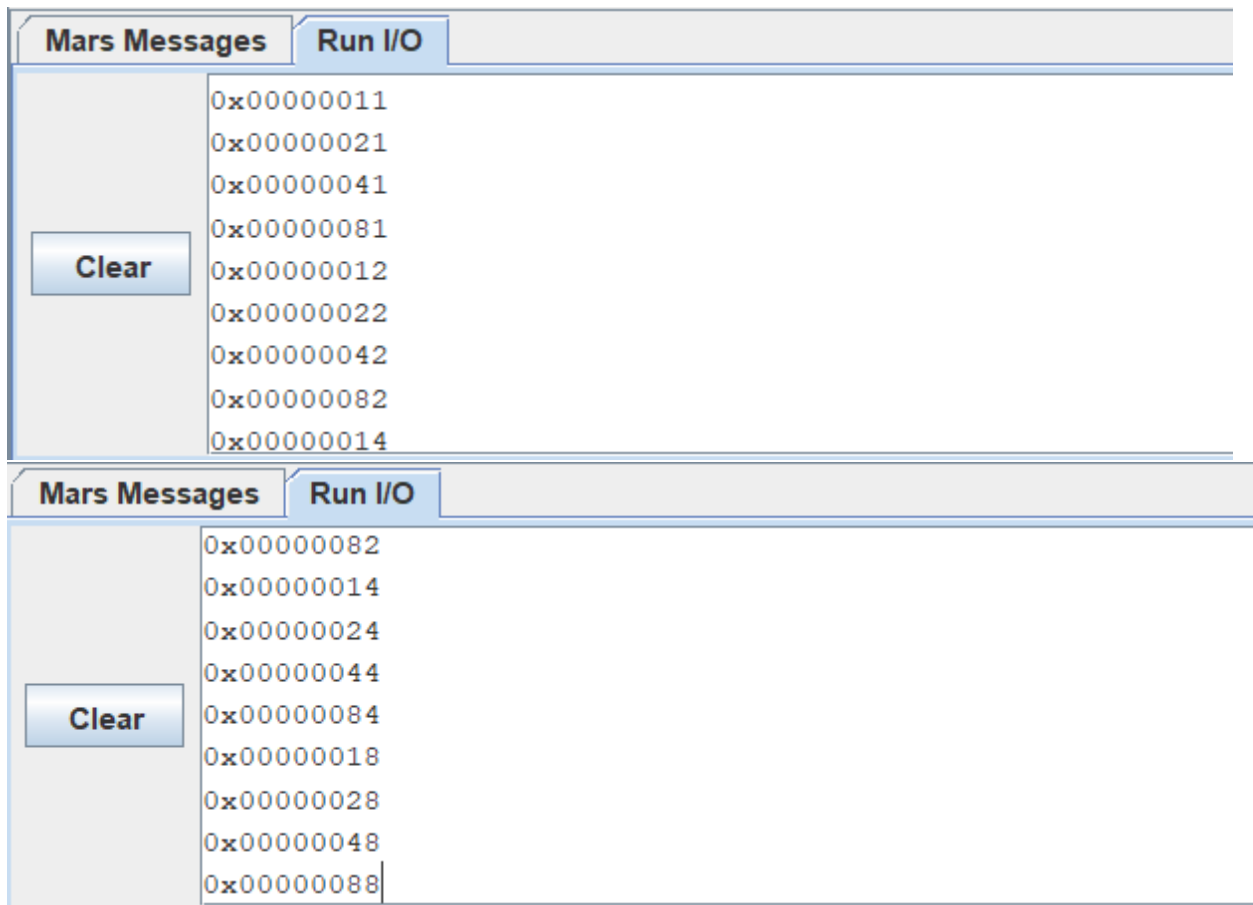
- Bấm phím 1



- Bấm phím 2



- Và lần lượt như vậy, được kết quả là



Assignment 2

- Code

```
.eqv IN_ADRESS_HEXА_KEYBOARD 0xFFFF0012
```

```
.eqv OUT_ADRESS_HEXА_KEYBOARD 0xFFFF0014
```

```
.data
```

```
Message1: .asciiz "Oh my god. Someone's presed a button: "
```

```
Message2: .asciiz "\n"
```

```
#~~~~~
```

```
# MAIN Procedure
```

```
#~~~~~
```

```
.text
```

```
main:
```

```
#-----
```

```
# Enable interrupts you expect
```

```
#-----
```

```
# Enable the interrupt of Keyboard matrix 4x4 of Digital Lab Sim
```

```
li $t1, IN_ADRESS_HEXА_KEYBOARD
```

```
li $t7, 0x80 # bit 7 of = 1 to enable interrupt
```

```
sb $t7, 0($t1)
```

```
#-----
```

```
# No-end loop, main program, to demo the effective of interrupt
```

```
#-----
```

```
Loop: nop
```

```
nop
```

```
nop
```

```
nop
```

b Loop # Wait for interrupt

end_main:

#~~~~~

GENERAL INTERRUPT SERVED ROUTINE for all interrupts

#~~~~~

.ktext 0x80000180

#-----

Processing

#-----

addi \$v0, \$zero, 4 # show message

la \$a0, Message1

syscall

li \$t1, IN_ADRESS_HEX_KEYBOARD

li \$t2, OUT_ADRESS_HEX_KEYBOARD

li \$t3, 0x01 # check row 4 with key 0, 1, 2, 3

li \$t4, 0x02 # check row 4 with key 4, 5, 6, 7

li \$t5, 0x04 # check row 4 with key 8, 9, A, B

li \$t6, 0x08 # check row 4 with key C, D, E, F

sb \$t3, 0(\$t1) # must reassign expected row

lbu \$a0, 0(\$t2) # read scan code of key button

bne \$a0, \$zero, print

sb \$t4, 0(\$t1) # must reassign expected row

lbu \$a0, 0(\$t2) # read scan code of key button

bne \$a0, \$zero, print

sb \$t5, 0(\$t1) # must reassign expected row

```

lbu $a0, 0($t2) # read scan code of key button
bne $a0, $zero, print
sb $t6, 0($t1) # must reassign expected row
lbu $a0, 0($t2) # read scan code of key button
bne $a0, $zero, print

```

print:

```

li $v0, 34 # print integer (hexa)
syscall

```

```

addi $v0, $zero, 4 # show message
la $a0, Message2
syscall

```

```

li $t7, 0x80 # bit 7 of = 1 to enable interrupt
sb $t7, 0($t1)

```

#-----

Evaluate the return address of main routine

epc <= epc + 4

#-----

next_pc:mfc0 \$at, \$14 # \$at <= Coproc0.\$14 = Coproc0.epc

addi \$at, \$at, 4 # \$at = \$at + 4 (next instruction)

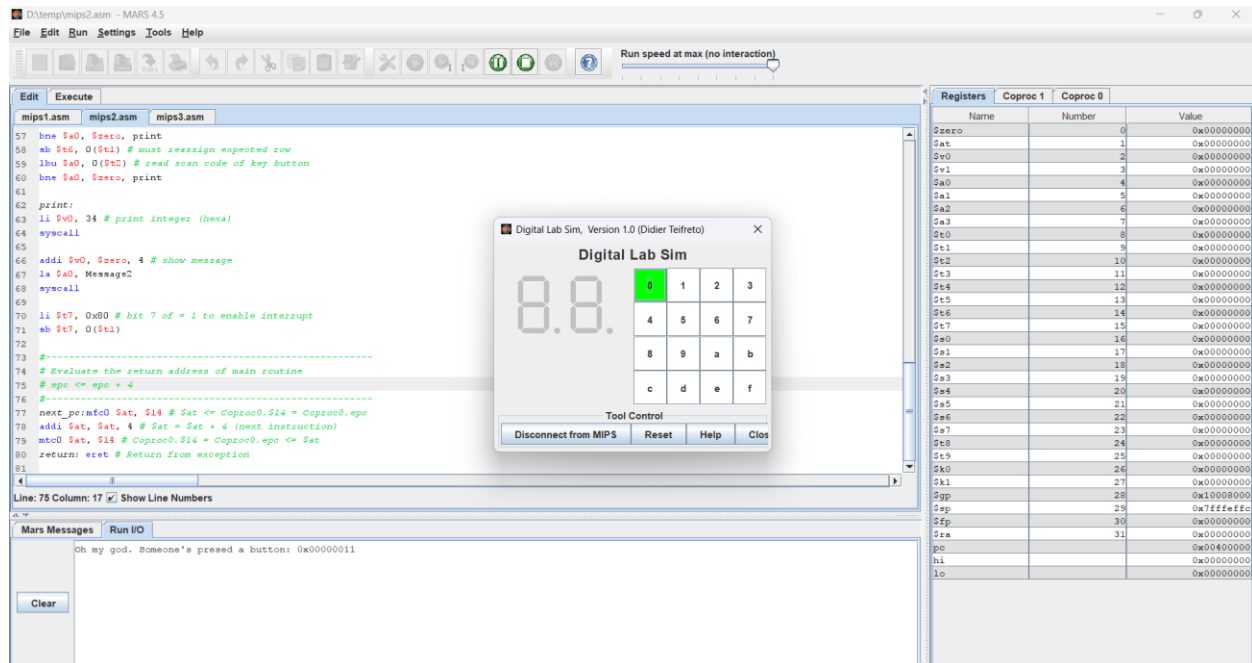
mtc0 \$at, \$14 # Coproc0.\$14 = Coproc0.epc <= \$at

return: eret # Return from exception

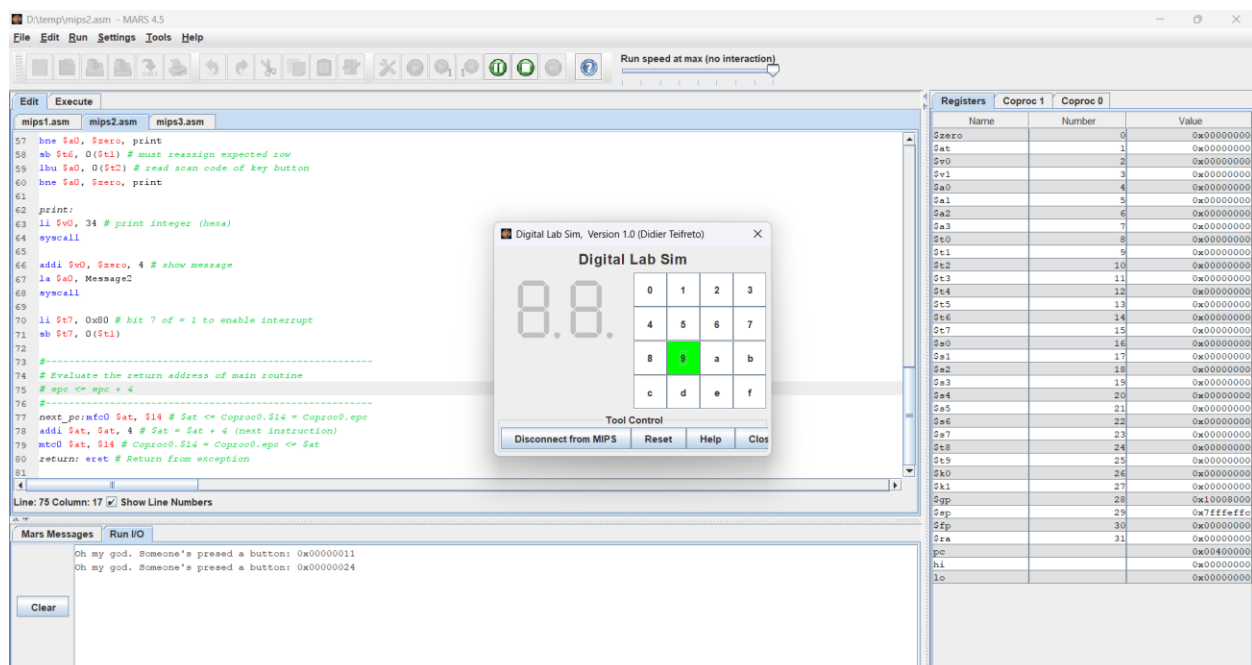
- Thực thi

-Chương trình thực hiện kiểm tra phím được bấm tương tự như bài 1.
Sau khi in ra kí tự được bấm thì gán giá trị ở
IN_ADRESS_HEX keyboard = 0x80 để enable interrupt.

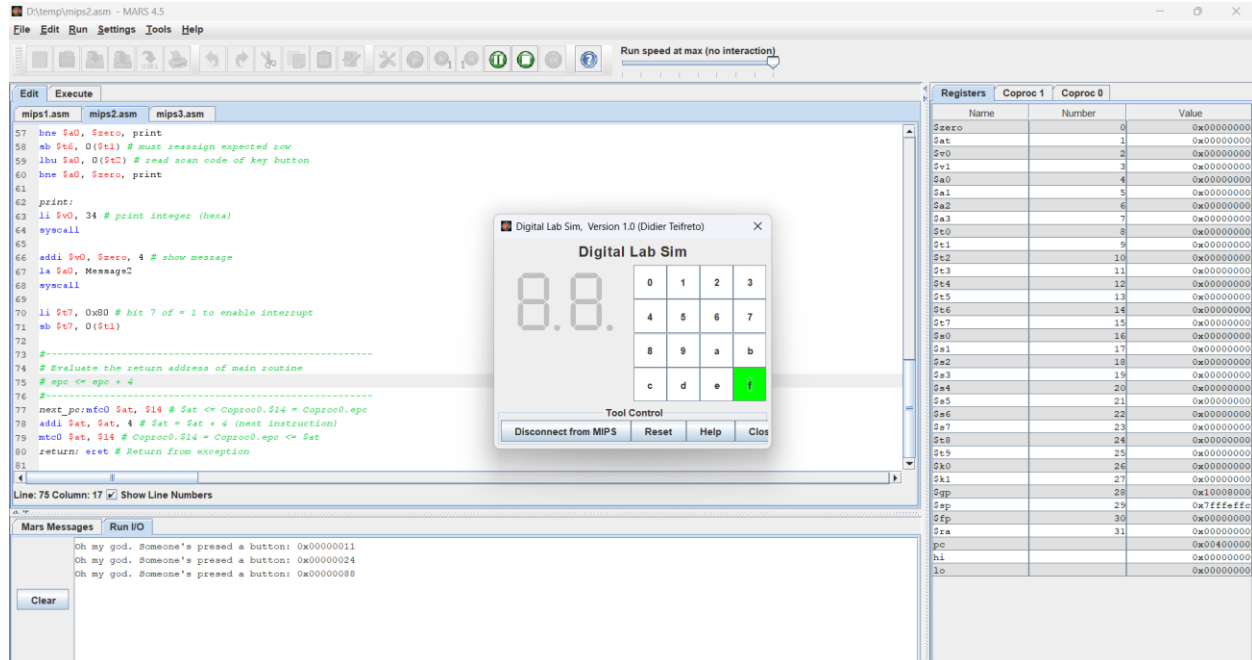
-Bấm phím 0



-Bấm phím 9



-Bấm phím f



Assignment 3

- Code

.eqv IN_ADDRESS_HEXKEYBOARD 0xFFFF0012

.eqv OUT_ADDRESS_HEXKEYBOARD 0xFFFF0014

.data

Message: .asciiz "Key scan code "

~~~~~

MAIN Procedure

~~~~~

.text

main:

Enable interrupts you expect

Enable the interrupt of Keyboard matrix 4x4 of Digital Lab Sim

```

li $t1, IN_ADRESS_HEX_A_KEYBOARD
li $t2, OUT_ADRESS_HEX_A_KEYBOARD
li $t3, 0x80 # bit 7 = 1 to enable
sb $t3, 0($t1)
#-----
# Loop and print sequence numbers
#-----
xor $s0, $s0, $s0 # count = $s0 = 0
Loop: addi $s0, $s0, 1 # count = count + 1
prn_seq: addi $v0, $zero, 1
add $a0, $s0, $zero # print auto sequence number
syscall
prn_eol: addi $v0, $zero, 11
li $a0, '\n' # print end of line
syscall
sleep: addi $v0, $zero, 32
li $a0, 1000 # sleep 300 ms
syscall
nop # WARNING: nop is mandatory here.
b Loop # Loop
end_main:
#~~~~~
# GENERAL INTERRUPT SERVED ROUTINE for all interrupts
#~~~~~
.ktext 0x80000180
#-----
# SAVE the current REG FILE to stack
#-----

```

IntSR: addi \$sp,\$sp,4 # Save \$ra because we may change it later

sw \$ra,0(\$sp)

addi \$sp,\$sp,4 # Save \$ra because we may change it later

sw \$at,0(\$sp)

addi \$sp,\$sp,4 # Save \$ra because we may change it later

sw \$v0,0(\$sp)

addi \$sp,\$sp,4 # Save \$a0, because we may change it later

sw \$a0,0(\$sp)

addi \$sp,\$sp,4 # Save \$t1, because we may change it later

sw \$t1,0(\$sp)

addi \$sp,\$sp,4 # Save \$t2, because we may change it later

sw \$t2,0(\$sp)

addi \$sp,\$sp,4 # Save \$t3, because we may change it later

sw \$t3,0(\$sp)

addi \$sp,\$sp,4 # Save \$t4, because we may change it later

sw \$t4,0(\$sp)

addi \$sp,\$sp,4 # Save \$t5, because we may change it later

sw \$t5,0(\$sp)

addi \$sp,\$sp,4 # Save \$t6, because we may change it later

sw \$t6,0(\$sp)

#-----

Processing

#-----

prn_msg: addi \$v0, \$zero, 4

la \$a0, Message

syscall

get_cod: li \$t1, IN_ADRESS_HEX_A_KEYBOARD

li \$t2, OUT_ADRESS_HEX_A_KEYBOARD

```

li $t3, 0x01 # check row 1 with key 0, 1, 2, 3
li $t4, 0x02 # check row 2 with key 4, 5, 6, 7
li $t5, 0x04 # check row 3 with key 8, 9, A, B
li $t6, 0x08 # check row 4 with key C, D, E, F
sb $t3, 0($t1 ) # must reassign expected row
lbu $a0, 0($t2) # read scan code of key button
bne $a0, $zero, prn_cod

sb $t4, 0($t1 ) # must reassign expected row
lbu $a0, 0($t2) # read scan code of key button
bne $a0, $zero, prn_cod

sb $t5, 0($t1 ) # must reassign expected row
lbu $a0, 0($t2) # read scan code of key button
bne $a0, $zero, prn_cod

sb $t6, 0($t1 ) # must reassign expected row
lbu $a0, 0($t2) # read scan code of key button
bne $a0, $zero, prn_cod

prn_cod:li $v0,34
syscall
li $v0,11
li $a0,'\n' # print endofline
syscall

#-----
# Evaluate the return address of main routine
# epc <= epc + 4
#-----

next_pc:mfc0 $at, $14 # $at <= Coproc0.$14 = Coproc0.epc
addi $at, $at, 4 # $at = $at + 4 (next instruction)
mtc0 $at, $14 # Coproc0.$14 = Coproc0.epc <= $at

```

```

#-----
# RESTORE the REG FILE from STACK
#-----

restore: li $t7, 0x80 # bit 7 of = 1 to enable interrupt
sb $t7, 0($t1)

lw $t6, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $t5, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $t4, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $t3, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $t2, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $t1, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $a0, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $v0, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $at, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

lw $ra, 0($sp) # Restore the registers from stack
addi $sp,$sp,-4

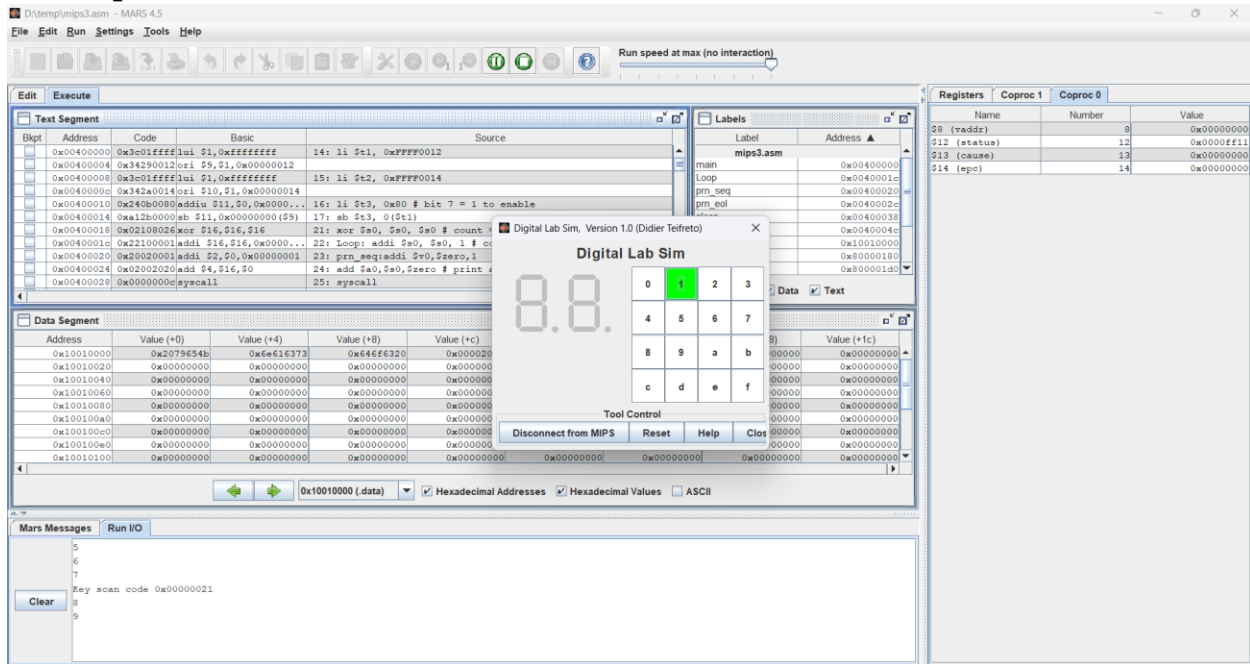
return: eret # Return from exception

```

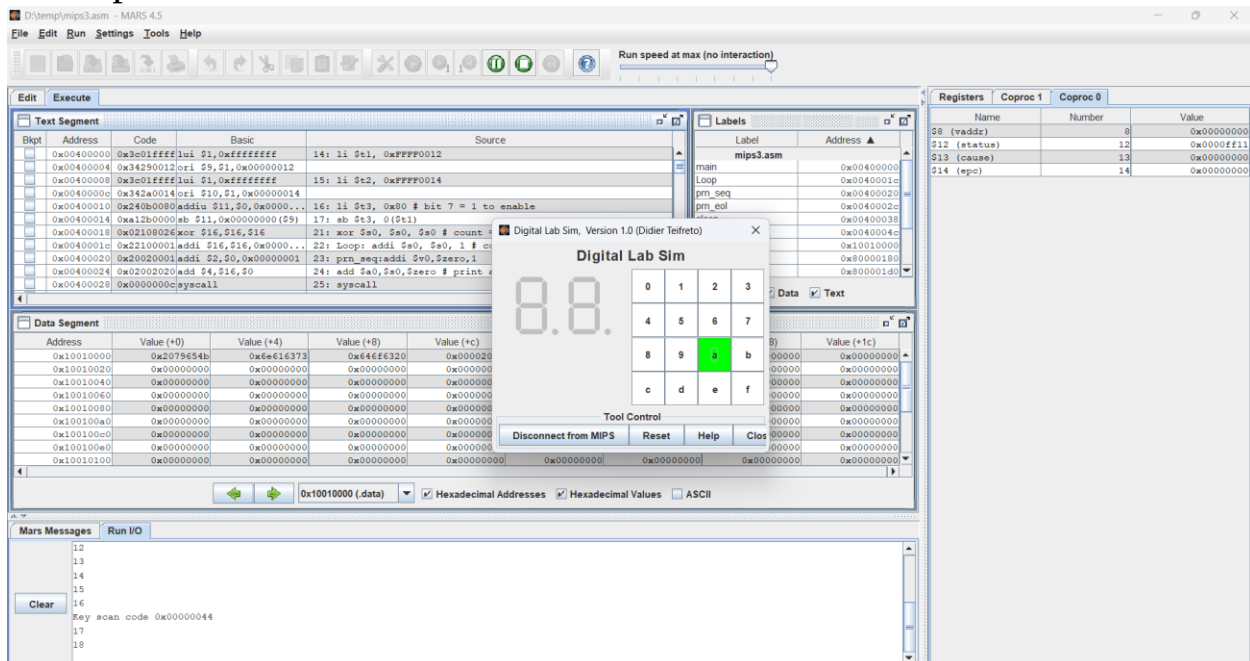
- Thực thi

-Chương trình kiểm tra nút được bấm tương tự như bài 2. Sau khi in ra kí tự được bấm thì gán giá trị ở IN_ADDRESS_HEX keyboard = 0x80 để enable interrupt.

-Bấm phím 1



-Bấm phím a



-Bấm phím d

The screenshot shows the MARS MIPS simulator interface. The main window displays the assembly code and its execution. The 'Digital Lab Sim' window is open, showing the number 8.8. The 'Registers' window on the right shows the status of various registers, including \$zero, \$at, \$v0, \$v1, \$a0, \$a1, \$a2, \$a3, \$t0, \$t1, \$t2, \$t3, \$t4, \$t5, \$t6, \$t7, \$s0, \$s1, \$s2, \$s3, \$s4, \$s5, \$s6, \$s7, \$s8, \$s9, \$k0, \$k1, \$gp, \$sp, \$fp, \$ra, \$pc, \$hi, and \$lo.

Register	Name	Number	Value
\$zero		0	0x00000000
\$at		1	0x00000000
\$v0		2	0x00000000
\$v1		3	0x00000000
\$a0		4	0x00000000
\$a1		5	0x00000000
\$a2		6	0x00000000
\$a3		7	0x00000000
\$t0		8	0x00000000
\$t1		9	0x00000000
\$t2		10	0x00000000
\$t3		11	0x00000000
\$t4		12	0x00000000
\$t5		13	0x00000000
\$t6		14	0x00000000
\$t7		15	0x00000000
\$s0		16	0x00000000
\$s1		17	0x00000000
\$s2		18	0x00000000
\$s3		19	0x00000000
\$s4		20	0x00000000
\$s5		21	0x00000000
\$s6		22	0x00000000
\$s7		23	0x00000000
\$s8		24	0x00000000
\$s9		25	0x00000000
\$k0		26	0x00000000
\$k1		27	0x00000000
\$gp		28	0x00000000
\$sp		29	0x7ffff000
\$fp		30	0x00000000
\$ra		31	0x00000000
\$pc			0x00000000
\$hi			0x00000000
\$lo			0x00000000

-Bấm phím c

The screenshot shows the MARS MIPS simulator interface. The main window displays the assembly code and its execution. The 'Digital Lab Sim' window is open, showing the number 8.8. The 'Registers' window on the right shows the status of various registers, including \$zero, \$at, \$v0, \$v1, \$a0, \$a1, \$a2, \$a3, \$t0, \$t1, \$t2, \$t3, \$t4, \$t5, \$t6, \$t7, \$s0, \$s1, \$s2, \$s3, \$s4, \$s5, \$s6, \$s7, \$s8, \$s9, \$k0, \$k1, \$gp, \$sp, \$fp, \$ra, \$pc, \$hi, and \$lo.

Register	Name	Number	Value
\$zero		0	0x00000000
\$at		1	0x00000000
\$v0		2	0x00000000
\$v1		3	0x00000000
\$a0		4	0x00000000
\$a1		5	0x00000000
\$a2		6	0x00000000
\$a3		7	0x00000000
\$t0		8	0x00000000
\$t1		9	0x00000000
\$t2		10	0x00000000
\$t3		11	0x00000000
\$t4		12	0x00000000
\$t5		13	0x00000000
\$t6		14	0x00000000
\$t7		15	0x00000000
\$s0		16	0x00000000
\$s1		17	0x00000000
\$s2		18	0x00000000
\$s3		19	0x00000000
\$s4		20	0x00000000
\$s5		21	0x00000000
\$s6		22	0x00000000
\$s7		23	0x00000000
\$s8		24	0x00000000
\$s9		25	0x00000000
\$k0		26	0x00000000
\$k1		27	0x00000000
\$gp		28	0x00000000
\$sp		29	0x7ffff000
\$fp		30	0x00000000
\$ra		31	0x00000000
\$pc			0x00000000
\$hi			0x00000000
\$lo			0x00000000